

# AI TRAINING – DAY 3 ASSIGNMENT

## Python Operators & Strings: Manipulate → Extract → Clean → Format

*Practical + MCQ + Debugging + String Utility Project*

### 1. Learning Goals

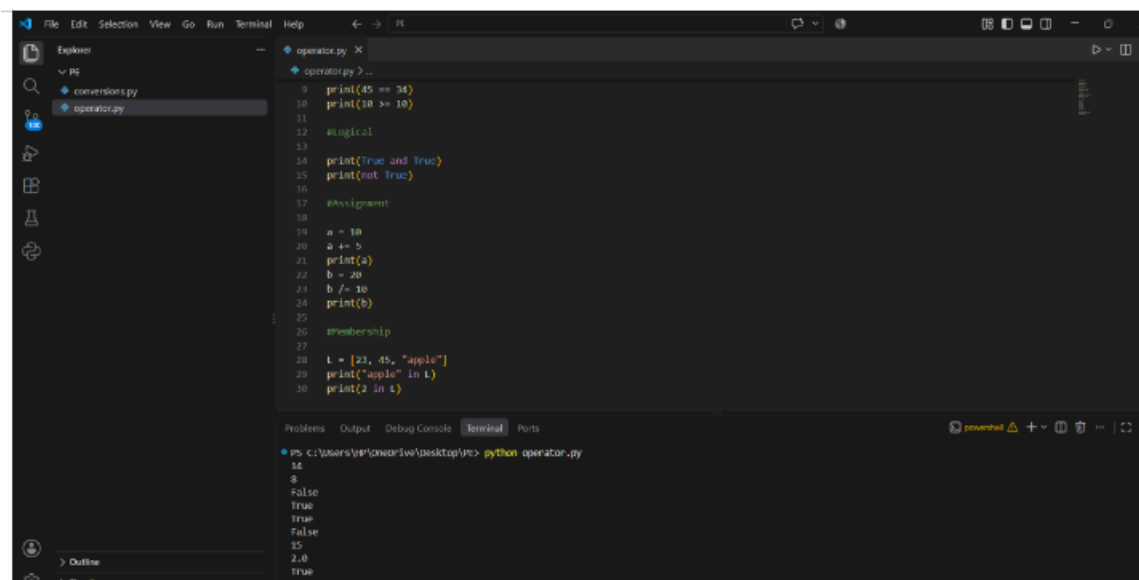
- Understand arithmetic, comparison, logical, assignment and membership operators.
- Understand strings, indexing, slicing and common string methods.
- Use f-strings to create readable dynamic output.
- Identify common operator and string errors.
- Combine all topics to build a practical text utility.
- Explain your solution in your own words.

### 2. Assignment Rules

- Type the programs yourself. Do not copy complete solutions.
- Before running code, predict the output.
- Test strings with spaces, uppercase/lowercase, empty input and unexpected input.
- You may use AI to explain a concept or error after attempting it yourself.
- For every practical problem write Input → Processing → Output.
- Keep at least one error you faced and explain how you fixed it.

### 3. Part A – Operators Exploration

Create operators.py and experiment with each group. Write at least 2 examples for every group.



```
9 print(45 == 34)
10 print(10 >= 10)
11
12 #logical
13
14 print(True and True)
15 print(not True)
16
17 #assignment
18
19 a = 10
20 a += 5
21 print(a)
22 b = 20
23 b /= 10
24 print(b)
25
26 #membership
27
28 l = [23, 45, "apple"]
29 print("apple" in l)
30 print(2 in l)
```

Output:

```
14
9
False
True
True
False
15
2.0
True
```

| Operator group | Operators / your observation                          |
|----------------|-------------------------------------------------------|
| Arithmetic     | + - * / // % **<br>Observation: Used for mathematical |

|            |                                                                                                                                            |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|            | calculations.Ex:10+4=14,<br>2**3=8                                                                                                         |
| Comparison | == != > < >= <=<br>Observation: used to compare values and return True or False. :45==34->False, 10>=10->True                              |
| Logical    | and or not<br>Observation: Used to compare conditions .True and True-> True, not True->False                                               |
| Assignment | = += -= *= /=<br>Observation: Used for assign values to variables and update them. Ex: a=10,_ b=20 b/=10 ->2                               |
| Membership | in / not in<br>Observation: Used to check a value is present in sequence or not .EX: L=[23,45,"apple"]<br>"apple" in L->True,2 in L->False |

## 4. Part B – Operator Thinking

1. What is the difference between / and //?

ANSWER: / It represents the division and it gives output as a float . EX:10/3->3.33

// It represents the floor division and gives output is always integer value of division.EX:10//3->3

2. When would % be useful in a real application?

ANSWER : % is used to find the remainder after division .It can be used to check whether a number is even or odd

Ex:10%2->0

3. What does \*\* do?

ANSWER: calculate the power of a number.

Ex:5\*\*2=25

4. What is the difference between = and ==?

ANSWER: = It is used to assign values to variables .Ex: a=5

== It is an comparison operator used to compare two values.Ex:4==5->False

5. What is the difference between != and not?

ANSWER: != is an comparison used to check two values are equal or not.Ex:3!=3->False

not is an logical operator used to reverse Boolean value .EX: not False->True

6. What is the result of 10 > 5 and 3 < 2? Explain.

ANSWER :10>5 output True and 3<2 output False .10 is greater then the 5 so true and 2 is less then 3 so flase.

7.What does not True produce?

ANSWER : not True produce False. not is an logical operator used to reverse Boolean value.

8..What does 'Python' in 'I am learning Python' return?

ANSWER : returns True .Because the 'python' exist in 'I am learning python'.

9.When would not in be useful?

ANSWER :not in will be useful when we check whether a value is exist in a string .

Example:- sen="my name is pavithra"

"Is" not in sen

## 5. Part C – String Basics Lab

Use your own name or another sentence and run these experiments.

1. Create a string and print it

```
ANSWER : name="pavithra"
        print(name)
        output: pavithra
```

2. Print its length using len().

```
ANSWER : name="pavithra"
        length=len(name)
        print(length)
        output:8
```

3. Print the first character using [0].

```
ANSWER : name="Pavithra"
        print(name[0])
        output: P
```

4. Print the last character using [-1].

```
ANSWER : name="Pavithra"
        print(name[-1])
        output:a
```

5. Print the first 3 characters using slicing.

```
ANSWER : name="Pavithra"
        print(name[0:3])
        output: Pav
```

6. Print the last 3 characters using slicing

```
ANSWER : name="Pavithra"
        Print(name[-3:])
        Output: hra
```

7. Print every second character.

```
ANSWER : name="Pavithra"
        Print(name[::2])
        Output: aiha
```

7. Reverse the string using slicing.

```
ANSWER : name="Pavithra"
        Print(name[::-1])
        Output: arhtivap
```

8. Try an invalid index. What error occurs?

```
ANSWER : name="Pavithra"
        Print(name[11])
        IndexError: string index out of range
```

9. Try changing one character directly. What happens and why?

```
ANSWER : name="Pavithra"
        name[2]="i"
        Print(name)
        Output: TypeError : 'str' object does not support item assignment
```

## 6. Part D – String Methods Lab

Experiment with these methods and explain what each one does.

| Method       | Your example                                                                                                             | What does it do?                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| lower()      | "PAvithRA".lower()->"pavithra"                                                                                           | Converts upper case letters into lower case letters                 |
| upper()      | "pAviTHra".upper()->"PAVITHRA"                                                                                           | Converts lower case letters into upper case letters                 |
| title()      | "Pavithra BILLupati".title()->"Pavithra Billupati"                                                                       | Title() converts the first letter of each word into capital letter. |
| strip()      | " pavith ra ".strip()->"pavithra"                                                                                        | Remove spaces from beginning and ending of a string                 |
| replace()    | Word=" I like pavithra".<br>replace("Pavan", "pavithra")                                                                 | Replace one part of string with another                             |
| split()      | "pavithra Billupati is very good person".split(" ")<br>Output: ["pavithra", "Billupati", "is", "very", "good", "person"] | Split the string into list                                          |
| join()       | " ".join(["a", "b", "c"])->"a b c"                                                                                       | Join elements into string                                           |
| startswith() | "Python".startswith("Py")<br>True                                                                                        | Check whether the string start with particular value                |
| endswith()   | "Harshitha".endswith("tha")<br>True                                                                                      | Check whether the string ends with particular value                 |
| find()       | "Python".find("t")<br>2                                                                                                  | Find particular value index position in string                      |
| count()      | "python program".count("p")<br>2                                                                                         | Count how many times a particular value occurs in a string          |

## 7. Part E – Indexing & Slicing Challenge

Use: text = "Artificial Intelligence"

1. Print the first character.

ANSWER :Print(text[0])->A

2. Print the last character.

ANSWER :print(text[-1])->e

3. Print the first 10 characters.

ANSWER :print(text[:10])->Artificial

4. Extract the word Intelligence.

ANSWER :print(text[11:])

5. Reverse the complete string.

ANSWER :Print(text[::-1])-> ecnegilletnl laicifitrA

6. Print every second character.

ANSWER :print(text[::2])-> Atfca nelgne

7. Print characters from index 3 to 12.

ANSWER: print(text[3:13])-> ificial In

8. Find the index of the first 'i'.

ANSWER :print(text.index("i"))->3

9. Count the number of 'i' character.

ANSWER :print(text.count("i"))->4

10. Predict what text[100] will do, then run it.

ANSWER : print(text[100])

IndexError: string index out of range

## 8. Part F – f-Strings

1.Create name, age and city variables and display them in one sentence using an f-string.

ANSWER:

```
name = "Pavithra"
age = 21
city = "Nellore"
print(f"My name is {name}, I am {age} years old, and I live in {city}.")
```

Output:

My name is Pavithra, I am 21 years old, and I live in Nellore

2.Create a product, price and quantity and display a formatted bill.

ANSWER :

```
product="phone"
price=200
quantity=4
bill=price*4
print(f"the cost of {quantity} {product} is {bill}")
```

Output:

the cost of 4 phone is 800

3.Display student name, total marks and average.

ANSWER :

```
student_name = "Pavithra"
total_marks = 425
average = 85
print(f"Student Name: {student_name}")
print(f"Total Marks: {total_marks}")
print(f"Average: {average}")
```

output:

Student Name: Pavithra

Total Marks: 425

Average: 85

4.Use an f-string to display a calculation result.

ANSWER :

```
a = 20
```

```
b = 10  
  
result = a + b  
  
print(f"The sum of {a} and {b} is {result}.")
```

output: The sum of 20 and 10 is 30.

5. Research decimal formatting and display a price with exactly 2 decimal places.  
ANSWER :

```
price = 99.5  
  
print(f"The price is {price:.2f}")
```

output: The price is 99.50

## 9. Part G – MCQs: Basic

1. Which operator means multiplication?

- A. x
- **B. \*** ✓
- C. %
- D. //

2. What does % return?

- A. Division
- B. Quotient
- **C. Remainder** ✓
- D. Power

3. What is the first string index?

- **A. 0** ✓
- B. 1
- C. -1
- D. first

4. What is len('Python')?

- A. 5
- **B. 6** ✓
- C. 7
- D. Error

5. Which gets the last character?

- A. text[0]
- **B. text[-1]** ✓

- C. text[last]
- D. text[1]

6. Which converts text to lowercase?

- **A. lower()** ✓
- B. small()
- C. down()
- D. lowercase

7. Which removes leading/trailing spaces?

- A. trim()
- B. clean()
- **C. strip()** ✓
- D. remove()

8. Which checks membership?

- A. inside
- **B. in** ✓
- C. has
- D. contains

9. Which is an f-string?

- **A. f"Hello {name}"** ✓
- B. "f Hello"
- C. format f(name)
- D. string(name)

10. Which compares equality?

- A. =
- **B. ==** ✓
- C. ===
- D. equals

## 10. Part H – MCQs: Medium

1. What is 17 // 5?

- **A. 3** ✓
- B. 3.4
- C. 2
- D. 4

2. What is 17 % 5?

- A. 3
- **B. 2** ✓
- C. 3.4

- D. 0

3. What does 'Python'[1:4] return?

- A. Pyt
- **B. yth ✓**
- C. ytho
- D. tho

4. What does 'Python'[-3:] return?

- A. tho
- **B. hon ✓**
- C. yth
- D. Python

5. What does ' hello '.strip() return?

- A. ' hello'
- B. 'hello '
- **C. 'hello' ✓**
- D. Error

6. What does 'hello'.upper() return?

- A. hello
- **B. HELLO ✓**
- C. Hello
- D. Error

7. What does 'a,b,c'.split(',') return?

- A. abc
- **B. ['a','b','c'] ✓**
- C. ('a','b','c')
- D. Error

8. Which checks if AI occurs in text?

- A. text.contains('AI')
- **B. 'AI' in text ✓**
- C. text.has('AI')
- D. contains(text,'AI')

9. What happens with text[50] when text has 10 characters?

- A. None
- B. ""
- **C. IndexError ✓**
- D. False

10. Why use f-strings?

- **A. Insert values/expressions into strings cleanly ✓**
- B. Make strings immutable
- C. Convert values to int
- D. Remove spaces

## 11. Part I – MCQs: Thinking

1. Which is true about strings?

- A. They can always be changed character-by-character
- **B. They are immutable ✓**
- C. They only contain letters
- D. They cannot be sliced

2. Best way to clean ' PHANI gorantla ' for display?

- A. strip().title()
- B. title().strip()
- **C. Both A and B ✓**
- D. clean()

3. What does text[::-1] do?

- A. Deletes text
- **B. Reverses text ✓**
- C. Sorts text
- D. Gets first character

4. Best simple test for '@' in an email?

- **A. '@' in email ✓**
- B. email == '@'
- C. email.contains('@')
- D. email.has('@')

5. replace() generally:

- **A. Returns a transformed string ✓**
- B. Always changes the original string in place
- C. Converts to list
- D. Deletes the variable

6. x='5', y=2. Which gives numeric 10?

- A. x\*2
- **B. int(x)\*2 ✓**
- C. str(x)\*2
- D. x+2

7. Which formats 12.5 as 12.50?

- **A. f'{price:.2f}' ✓**

- B. f'{price:2}'
- C. f'{price.float}'
- D. f'{price.decimal(2)}'

8. What should usually happen before validating text entered with spaces?

- **A. strip()** ✓
- B. reverse()
- C. split() always
- D. replace with None

9. A common cause of IndexError is:

- **A. Invalid index** ✓
- B. Using an f-string
- C. lower()
- D. +

10. Why keep raw input and cleaned input conceptually separate?

- **A. It makes transformations easier to test** ✓
- B. Python requires it
- C. Strings cannot be stored
- D. It prevents every error

## 12. Part J – 5 Practice Problems

| Problem                             | Task                                                                                                                 | Topics                        |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Practice 1 – Username Cleaner       | Ask for a username, remove surrounding spaces, convert to lowercase and display the cleaned value.                   | strip + lower + f-string      |
| Practice 2 – Name Formatter         | Ask first and last names in random case and display a properly formatted full name.                                  | strip + title + f-string      |
| Practice 3 – Word Analyzer          | Ask for a sentence. Display length, first character, last character, spaces count and vowel count. Do not use loops. | len + indexing + count        |
| Practice 4 – Email Inspector        | Ask for an email. Check '@', check whether it ends with '.com' or '.in', and show the position of '@'.               | in + endswith + find          |
| Practice 5 – Product Code Extractor | For code IND-MOB-2026-001, extract country, category, year and serial using slicing.                                 | indexing + slicing + f-string |

For each: Input → Expected Output → Code → 2 test cases → 1 edge case → explanation.

### Practice 1 – Username Cleaner:

#### Code:

```
username=input("Enter the username :")
```

```
username1=username.strip()
```

```
low_username=username1.lower()
print(f"the user name is {low_username}")
```

**output:**

Enter the username :PaviTHra

the user name is pavithra

**Practice 2 – Name Formatter:**

**CODE:**

```
user_frist_name=input("enter the frist name:")
user_last_name=input("enter the last name:")
frist_name=user_frist_name.lower()
last_name=user_last_name.lower()
print(f"The full name of user is {frist_name} {last_name}")
```

**OUTPUT:**

enter the frist name:pavltHra

enter the last name:BiLLupati

The full name of user is pavithra billupati

**Practice 3 – Word Analyzer:**

**Code:**

```
sentence=input("enter any sentence :")
length_sen=len(sentence)
frist_char=sentence(0)
last_char=sentence(-1)
spec_count=sentence.count(" ")
vowel=(sentence.lower().count("a")
+sentence.lower().count("e")
+ sentence.lower().count("i")
+ sentence.lower().count("o")
+ sentence.lower().count("u"))
print(f"length of sentence is {length_sen}")
print(f"the frist character of sentence is {frist_char}")
```

```
print(f"last charater in the sentence is {last_char}")  
print(f"{spec_count} of spaces in the sentences")  
print(f"{vowel} of vowels in the sentences")
```

**output:**

```
enter any sentence :pavithra very good person  
length of sentence is 25  
the first character of sentence is p  
last character in the sentence is n  
3 of spaces in the sentences  
8 of vowels in the sentences
```

**Practice 4 – Email Inspector:**

**CODE:**

```
mail_id=input("Enter the mail :")  
is_endwith=mail_id.endswith(".com")  
print("it ends with @","@" in mail_id,)  
print("it end with .com",is_endwith)  
print("at",mail_id.find("@"),"index @ is exisis ")
```

**output:**

```
Enter the mail :pavithra@gmail.com  
it ends with @ True  
it end with .com True  
at 8 index @ is exist
```

**Practice 5 – Product Code Extractor:**

**CODE:**

```
product_code=input("enter the product code like this counter-category-year-serial(IND-MOB-2026-001) :")  
country=product_code[:3]  
category=product_code[4:7]  
Year=product_code[8:12]  
serial=product_code[13:]  
print(f"country {country}")
```

```
print(f'category {category}')
```

```
print(f'year {Year}')
```

```
print(f'serial {serial}')
```

**output:**

enter the product code like this counter-category-year-serial(IND-MOB-2026-001) :USA-LAP-2026-001

country USA

category LAP

year 2026

serial 001

## 13. Part K – Debugging Lab

### Bug 1

```
name = input('Name: ')
print(name[20])
```

Why can this fail? Fix it.

Corrected code / explanation:

```
name = input('Name: ')
print(name)
```

Explanation: We should not access a fixed index if we don't know that the string length.

### Bug 2

```
age = '20'
print(f'Age: {age + 1}')
```

Fix it.

Corrected code / explanation:

```
age = '20'
print(f'Age: {int(age) + 1}')
```

Explanation: age is a variable store the string value if we try to add the string value with 1 it produce error.so we convert the age string variable value into integer value using int()

### Bug 3

```
name = 'phani'
age = 30
print('Hello ' + name + age)
```

Identify the type problem and fix it.

Corrected code / explanation:

```
name = 'phani'
```

```
age = 30
```

```
print(f'Hello {name}, you are {age} years old.')
```

Explanation: This is a type mismatch. An f-string allows us to display both the string and integer easily.

#### Bug 4

```
email = input('Email: ')
print(email.endswith('.com'))
```

Test with 3 inputs and explain.

Corrected code / explanation:

```
email = input('Email: ')

print(email.endswith('.com'))
```

#### Bug 5

```
code = 'IND-MOB-2026-001'
print(code[20])
```

Find the valid index range and fix the requirement.

Corrected code / explanation:

```
code = 'IND-MOB-2026-001'

print(code[16])
```

Explanation: code variable has only length 16 but we print code[20] it causes indexerror

#### Bug 6

```
text = ' hello world '
print(text.upper().strip())
```

Does it work? Explain the order of operations.

Corrected code

```
text = ' hello world '
print(text.upper().strip())
```

Explanation: Operation is performed from left to right .First text is converted into upper case letters. Then remove the spaces from the beginning and end.

## 14. Part L – Main Project: Text-Cleaning & Profile Formatter

Build a practical Python program that accepts messy profile information and converts it into a clean, readable profile.

### Scenario

A company receives profile data in inconsistent formats. Names may contain extra spaces, emails may use mixed cases, and cities may be entered randomly. Your program should clean the data and generate a professional profile summary.

## Required Inputs

- Full name
- Email address
- Phone number
- City
- Job role
- Short bio

## Must-Have Requirements

- Remove leading/trailing spaces.
- Format the full name using title().
- Convert email to lowercase.
- Format city and job role neatly.
- Calculate bio character count.
- Check whether the email contains '@'.
- Check whether the email ends with a supported domain such as .com or .in.
- Extract first name from the full name.
- Extract last name using string logic.
- Display the profile using f-strings.
- Show a small data-quality section.
- Test with messy uppercase/lowercase and extra spaces.

## Expected Output Example

```
===== PROFILE =====
Name       : Phani Gorantla
First Name : Phani
Email      : phani@example.com
Phone     : 9876543210
City      : Hyderabad
Role      : Software Developer
Bio Length : 42 characters

----- DATA QUALITY -----
Email Valid : Yes
=====
```

Do not hard-code the example. The program must work with any user's input.

### CODE:

```
full_name=input("enter your full name:")
```

```
Email=input("enter your email:")
```

```
phone=input("enter your number:")
```

```
city=input("enetr your city name:")
```

```
jobrole=input("enter your job role:")
```

```
bio=input("enter your short bio:")
```

```
full_name=full_name.strip()
```

```
Email=Email.strip()
```

```
jobrole=jobrole.strip()
```

```
bio=bio.strip()
```

```
full_name=full_name.title()
```

```
Email=Email.lower()
```

```
city=city.strip().title()
```

```
jobrole=jobrole.strip().title()
```

```
bio_length=len(bio)
```

```
check="@ "in Email
```

```
end1=Email.endswith(".com")
```

```
end2=Email.endswith(".in")
```

```
first_name = full_name.split()[0]
```

```
last_name = full_name.split()[-1]
```

```
email_valid = check and (end1 or end2)
```

```
print(f"Name :{full_name}")
```

```
print(f"First Name :{first_name}")
```

```
print(f"Last name:{last_name}")
```

```
print(f"Email:{Email}")
```

```
print(f"phone:{phone}")
```

```
print(f"city:{city}")
```

```
print(f"Role:{jobrole}")
```

```
print(f"Bio_length:{bio_length}")
```

```
print("----- DATA QUALITY-----")
```

```
print(f"Email valid:{email_valid}")
```

### **OUTPUT:**

```
enter your full name: billupati pavithra
```

```
enter your email:pavithra@gmail.com
```

```
enter your number:123456789
```

enter your city name:nellore

enter your job role:software

enter your short bio :I am intrested in programming languages

Name :billupati pavithra

First Name :pavithra

Last name:billupati

Email:pavithra@gmail.com

phone:123456789

city:Nellore

Role:Software

Bio\_length:39

----- DATA QUALITY-----

Email valid: True

## 15. Project Bonus Challenges

- Clean repeated spaces inside a name.
- Validate phone number length.
- Check that the full name contains at least two words.
- Generate initials from the name.
- Generate a username from the person's name.
- Create a one-line professional bio using an f-string.
- Display a simple data-quality percentage.

### CODE:

```
full_name = input("Enter your full name: ")
email = input("Enter your email: ")
phone = input("Enter your phone number: ")
city = input("Enter your city: ")
role = input("Enter your job role: ")
bio = input("Enter your short bio: ")
full_name = " ".join(full_name.split())
phone_length = len(phone)
phone_valid = phone_length == 10
word_count = len(full_name.split())
name_valid = word_count >= 2

words = full_name.split()
initials = words[0][0].upper() + words[1][0].upper()
username = words[0].lower() + "_" + words[1].lower()
professional_bio = f"{full_name} is interested in {role} and programming."
filled = bool(full_name) + bool(email) + bool(phone) + bool(city) + bool(role) + bool(bio)
```

```

quality = (filled / 6) * 100
print("\n--- Bonus Challenge Results ---")
print("Clean Name:", full_name)
print("Phone Length:", phone_length)
print("Phone Valid:", phone_valid)
print("Word Count:", word_count)
print("Name Valid:", name_valid)
print("Initials:", initials)
print("Username:", username)
print("Professional Bio:", professional_bio)
print("Data Quality:", quality, "%")

```

### OUTPUT:

```

Enter your full name: biLluPATi pAVIthra
Enter your email: pavithra@gmail.com
Enter your phone number: 123456789
Enter your city: nellore
Enter your job role: developer
Enter your short bio: i am intresed in programming languages

```

```

--- Bonus Challenge Results ---
Clean Name: billupati pavithra
Phone Length: 10
Phone Valid: True
Word Count: 2
Name Valid: True
Initials: MH
Username: billupati_pavithra
Professional Bio: billupati pavithra is interested in developer and programming.
Data Quality: 100.0 %

```

## 16. Project Test Cases

### 16. Project Test Cases

| Test | Input condition             | Expected behaviour                | Actual result                                     |
|------|-----------------------------|-----------------------------------|---------------------------------------------------|
| 1    | Normal clean input          | Clean profile                     | Profile displayed corrected                       |
| 2    | Extra spaces around fields  | Spaces removed                    | Extra spaces removed                              |
| 3    | Random uppercase/lowercase  | Proper formatting                 | Name and username formatted properly              |
| 4    | Invalid-looking email       | Flag data quality                 | Data quality percentage shows the available data  |
| 5    | Short/missing optional data | Program should handle it sensibly | Program accepts the input and displays the result |

## 17. Explain Your Project

10. What is the difference between cleaning data and formatting data?

A) Data cleaning means removing unwanted spaces or incorrect data. Data formatting means presenting the data in a proper and consistent way.

11. Which string methods did you use and why?

A) I used `split()`, `join()`, `upper()`, and `lower()`.

- `split()` → it separates words.
- `join()` → it joins words together.
- `upper()` → converts the letters to uppercase.
- `lower()` → converts letters to lowercase.

12. Where did you use indexing?

A) I used indexing to get the first letter of the name for generating initials

```
EX: first_name = words[0]
    short_name = first_name[:3]
```

13. Where did you use slicing?

A) Slicing is used to print the substring from specified conditions

```
EX: first_name = words[0]
    short_name = first_name[:3]
```

14. Why are f-strings useful here?

A) F-strings are useful because they allow me to easily combine variables and text in one string.

```
EX: Print(f" my full name is :{name}")
```

15. What error did you face?

A) Initially faced errors while working with strings and spaces. I corrected them by using string methods such as `split()` and `join()`.

16. How did you test edge cases?

A) Tested the program with extra spaces, different uppercase and lowercase letters, invalid-looking email, short phone numbers, and missing data.

17. If this processed 1 lakh profiles, what would you improve first?

A) I would improve the data validation and processing speed.

## 18. Final Self-Check

| Skill                                       | Yes                        | Need Practice              | Notes |
|---------------------------------------------|----------------------------|----------------------------|-------|
| Use arithmetic/comparison/logical operators | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Explain = vs ==                             | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Manipulate strings                          | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Use positive and negative indexing          | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Use slicing                                 | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Use string methods                          | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Use in/not in                               | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Use f-strings                               | <input type="checkbox"/> ✓ | <input type="checkbox"/>   |       |
| Debug IndexError/type                       | <input type="checkbox"/>   | <input type="checkbox"/> ✓ |       |

|                              |                            |                          |  |
|------------------------------|----------------------------|--------------------------|--|
| problems                     |                            |                          |  |
| Build the final text utility | <input type="checkbox"/> ✓ | <input type="checkbox"/> |  |

## 19. Submission Checklist

- Operator experiments
- Operator questions
- String exploration
- String-method table
- Indexing/slicing challenge
- f-string exercises
- 30 MCQ answers
- 5 practice programs
- Debugging Lab
- Text-Cleaning & Profile Formatter
- Project test cases
- Project explanation and self-check